



PDF Download
3589334.3645407.pdf
25 December 2025
Total Citations: 19
Total Downloads: 1205

Latest updates: <https://dl.acm.org/doi/10.1145/3589334.3645407>

RESEARCH-ARTICLE

Trident: A Universal Framework for Fine-Grained and Class-Incremental Unknown Traffic Detection

ZIMING ZHAO, Zhejiang University, Hangzhou, Zhejiang, China

ZHAOXUAN LI, Institute of Information Engineering, Beijing, China

ZHUOXUE SONG, Zhejiang University, Hangzhou, Zhejiang, China

WENHAO LI, Institute of Information Engineering, Beijing, China

FAN ZHANG, Zhejiang University, Hangzhou, Zhejiang, China

Open Access Support provided by:

Zhejiang University

Institute of Information Engineering

Published: 13 May 2024

[Citation in BibTeX format](#)

WWW '24: The ACM Web Conference
2024

May 13 - 17, 2024
Singapore, Singapore

Conference Sponsors:
SIGWEB

Trident: A Universal Framework for Fine-Grained and Class-Incremental Unknown Traffic Detection

Ziming Zhao
Zhejiang University
Hangzhou, China
zhaoziming@zju.edu.cn

Zhaoxuan Li
Institute of Information Engineering,
CAS, Beijing, China
lizhaoxuan@iie.ac.cn

Zhuoxue Song
Zhejiang University
Hangzhou, China
songzhuoxue@zju.edu.cn

Wenhao Li
Institute of Information Engineering,
CAS, Beijing, China
liwenhao@iie.ac.cn

Fan Zhang*
Zhejiang University
Hangzhou, China
fanzhang@zju.edu.cn

ABSTRACT

To detect unknown attack traffic, anomaly-based network intrusion detection systems (NIDSs) are widely used in Internet infrastructure. However, the security communities realize some limitations when they put most existing proposals into practice. The challenges are mainly concerned with (i) fine-grained emerging attack detection and (ii) incremental updates/adaptations. To tackle these problems, we propose to decouple the need for model capabilities by transforming known/new class identification issues into multiple independent one-class learning tasks. Based on the above core ideas, we develop Trident, a universal framework for fine-grained unknown encrypted traffic detection. It consists of three main modules, *i.e.*, tSieve, tScissors, and tMagnifier are used for profiling traffic, determining outlier thresholds, and clustering respectively, each of which supports custom configuration. Using four popular datasets of network traces, we show that Trident significantly outperforms 16 state-of-the-art (SOTA) methods. Furthermore, a series of experiments (concept drift, overhead/parameter evaluation) demonstrate the stability, scalability, and practicality of Trident.

CCS CONCEPTS

• Security and privacy → Network security; • Information systems → Traffic analysis.

KEYWORDS

Fine-grained unknown traffic detection, class-incremental learning

ACM Reference Format:

Ziming Zhao, Zhaoxuan Li, Zhuoxue Song, Wenhao Li, and Fan Zhang. 2024. Trident: A Universal Framework for Fine-Grained and Class-Incremental Unknown Traffic Detection. In *Proceedings of the ACM Web Conference 2024 (WWW '24)*, May 13–17, 2024, Singapore, Singapore. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3589334.3645407>

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).
WWW '24, May 13–17, 2024, Singapore, Singapore

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0171-9/24/05...\$15.00
<https://doi.org/10.1145/3589334.3645407>

1 INTRODUCTION

Traffic analysis is an important mechanism for security investigation, such as network intrusion detection systems (NIDSs), malware identification, *etc.* With encrypted traffic transmission becoming ubiquitous in practice, the proposed approaches gradually evolve from signature-based [26] to machine-learning-based (ML-based) detection in the traffic analysis landscape [16]. These ML-based schemes aim to characterize traffic patterns with packet fields or sequence features since the transmission content is encrypted, and they are able to detect unknown attacks [12]. For instance, anomaly-based solutions construct profiles of benign traffic to discover unforeseen attacks that deviate from legitimate samples [11, 12, 28, 33, 35, 55]. Thus, anomaly detection becomes an indispensable step for security in the real world. However, academic communities and industrial practitioners reveal a series of limitations when they put most existing anomaly-based proposals into practice [8, 21, 28, 44]. By summarizing those issues, we recognize the following two main challenges.

(i) *Fine-grained unknown attack detection.* The anomaly-based methods can identify unknown attacks, while previous proposals are usually binary classification models [5, 33, 35]. That is to say, they could only infer whether the sample is “benign” or “abnormal”, but cannot recognize that the anomaly is “unknown attack 1”, “unknown attack 2”, or “unknown attack 3”, *etc.* Yet these fine-grained labels are the key for defenders to deploy effective countermeasures against the attacks [8, 28]. For example, the victims can count the per-flow protocol flag to mitigate flood-based DDoS [32, 59, 61, 62]. Also, they can defend the reflection-based attacks by source verification [40]. Therefore, the binary classification anomaly detection leads to a *semantic gap* between the model identification results and the actionable reports for network operators [44]. If the proposed scheme can automatically distinguish different unknown attack classes based on the network traffic characterization, it could facilitate understanding attack details and implementing corresponding countermeasures. Consequently, the first challenge is to detect the unknown (and known as well) attack in a fine-grained manner.

(ii) *Incremental update.* Incremental update requirements include *sample increments* and *class increments*. The former means the known-class traffic that could be ever-changing (also known as the concept drift problem [2, 5, 15, 22, 56]). In most anomaly-based detection, they advocate only using benign traffic to train the classifier, *i.e.*, “zero-positive” learning [5, 16], then those samples deviating

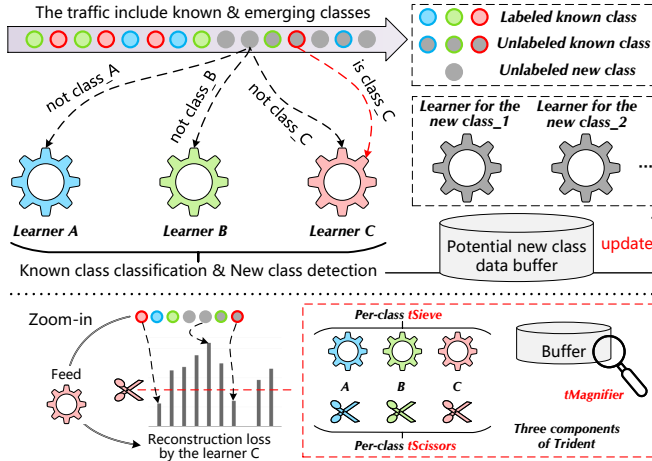


Figure 1: The workflow of Trident and three components.

from legitimate traffic will be considered malicious. In this way, it could appear a large number of false positives when the legitimate traffic manifested as different from priori properties. Another requirement “class increments” refers to detected emerging classes that should be incrementally updated into the model to become known classes in the follow-up. Therefore, it is non-trivial to attach new classes to the model’s knowledge base without affecting the previously known classes.

In this paper, we propose a universal framework, named as Trident¹, aiming to enable process three abilities, *i.e.*, the known class classification, fine-grained unknown class detection, and incremental model update (including *sample increments* and *class increments*). At the high level, Trident is designed with three tightly coupled components named tSieve, tScissors, and tMagnifier in Fig. 1 (notably, each component supports custom configuration to meet various users’ requirements).

First, tSieve maintains a series of one-class learners for known classes. Each single-class learner is built upon only one class of data. Taking the AutoEncoder as an example, the well-trained tSieve will output a smaller reconstruction loss for the same class samples, while outputting a large loss for other classes. Then, tScissors is responsible for deploying the outlier detection algorithm to automatically determine the threshold for each one-class learner, which decides whether per-sample is accepted or rejected by tSieve. If a sample is accepted by at least one learner in tSieve, it is considered a known class and tagged with the most similar label. Otherwise, those samples rejected by all learners will enter the buffer pool to further assign fine-grained new class labels through the tMagnifier clustering process.

The main benefit of such a framework is that it decouples the requirements of model capabilities, and each learner only needs to focus on one class of samples. In this way, when an emerging class arrives, only a new learner needs to be added without changing the existing learner. Meanwhile, model capability decoupling can mitigate the impact of concept drift to some extent. As shown in Fig. 2 (a), typical zero-positive learning fits a wide variety of

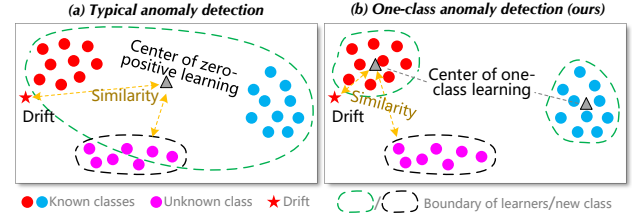


Figure 2: Illustration of coping with concept drift.

legitimate flows, forming a generalized sample center (marked as a grey triangle). This makes it difficult to distinguish drifts of benign samples from unknown attacks, since their similarity to the center is comparable. While the one-class learning of Trident is essentially a divide-and-conquer strategy, it has more distinguishable² for drift samples and unknown classes in Fig. 2 (b). Furthermore, if there is concept drift in any class, just incrementally update its learner and outlier threshold based on drift samples to complete the adaptation, *i.e.*, completing *sample increments*.

In summary, this paper makes three key contributions.

- We carefully examine the problems of current anomaly-based NIDSs in practice and summarize them as two key challenges. To facilitate addressing the above issues, we propose Trident, a universal framework for fine-grained traffic detection.
- We design three tightly coupled components (tSieve, tScissors, tMagnifier) for Trident, thereby realizing the known class classification, fine-grained unknown class detection, and incremental model update simultaneously. Notably, they all support the customized configuration.
- Through experiments, we demonstrate that Trident significantly outperforms previous methods. Meanwhile, we conduct a series of additional experiments to show its superior stability, scalability, and practicality.

2 ASSUMPTIONS AND PROBLEM SPACE

2.1 Threat Model and Assumptions

Adversary Model. We consider unknown intrusions such as zero-day attacks that exist in real-world scenarios. In other words, strong adversaries will adopt emerging attack strategies that are previously unseen by victims, including variants of the existing attacks or brand new ones. Thus, it is hard to have any prior data about these unforeseen attacks. Moreover, the traffic of known types is not set in stone. Therefore, the problem of concept drift is within the scope of consideration. In addition, we mainly focus on *encrypted traffic analysis* in this paper, since the transmission content is increasingly being encrypted in existing networks, such as SSL/TLS and SSH. Concretely, we tend to characterize traffic behavior by portraying packet field distribution rather than analyzing transmission content, *e.g.*, TCP Payload.

Assumptions. We assume there is no prior knowledge when suffering unprecedented attacks in practice. We are also not aware of how many types of unknown attacks exist in the collected traffic samples in advance. Meanwhile, we do not assume additional collaborations from other Internet entities, such as IP blacklists provided by security vendors. In addition, multiple unknown attacks may be

¹In ancient Greek mythology, Trident is the weapon of Poseidon, symbolizing great power. Our proposal is designed to perform powerful detection capabilities for known/unknown traffic.

²It refers to that for a one-class learner, the unknown-class instances tend further away from the single-class sample center than the drift sample.

launched at the same time or may appear alternately. That is to say, we do not make assumptions about the distribution of attacks.

2.2 Problem Formulation

We provide precise definitions of two critical challenges.

Fine-Grained Unknown Attack Detection. Given a prior dataset S_{tra} , consisting of samples from benign traffic B and known cyber attacks $\{A_k^1, A_k^2 \dots A_k^n\}$, where n refers to the number of known classes. And we use S_{tra} as the training set to fit the model M . When deploying M in practice, it will encounter the open-world testset include: (i) samples which the ground-truth labels from $\{B, A_k^1, A_k^2 \dots A_k^n\}$; (ii) instances of emerging classes $\{A_u^1, A_u^2 \dots A_u^m\}$, where m denotes the number of unknown classes, and m is unknown to us in advance. The *fine-grained unknown attack detection* refers to: M can identify the specific-attack labels of test samples, i.e., the sample prediction result is $B, A_k^1, A_k^2, A_u^1, A_u^2$, or others. More extended definitions are discussed in Appendix A.

Incremental Update. The requirements of incremental updates include two parts: *sample increments* and *class increments*. For the sample increment, it refers to the model adapting to new instances without retraining the whole architecture. This capability stems from two considerations, for one thing, the business scenario changes will cause different manifestations of legitimate traffic (e.g., the traffic of streaming media and chatting would be very different). For another, sample increments could also be used to adapt to the concept drift of attack (such as some attacks changing over time, making the previous attack traffic somewhat outdated).

Regarding the class increment, it is actually an inevitable problem for fine-grained detection, and it is also a major difference between us and existing work. When a new type of attack A_u^* is detected, we hope to add it to the knowledge base to strengthen the model's capability. In other words, the updated model can detect A_u^* just like supervised classification in the future. This is a challenging task that cannot be achieved by most existing works.

3 TRIDENT FRAMEWORK OVERVIEW

In this section, we depict a high-level workflow of Trident, including the key building blocks of each component. Most typical schemes are either supervised learning for multi-class classification or binary classification anomaly detection. We architecture Trident as a novel framework to decouple the requirements of model capabilities. The overall process includes three parts: one-class learning, outlier determination, and fine-grained label assignment.

One-Class Learning. First, unlike typical models, Trident maintains a collection of one-class learners, each of which corresponds to a traffic type one-to-one. Each learner is only responsible for reconstructing the corresponding type of traffic data, and the learners are independent of each other. Therefore, when a test sample arrives, each learner outputs its reconstruction loss and decides to accept or reject this instance. The instance that cannot be accepted by any learner is considered the new class, otherwise, it will be assigned as the most similar class among all accepted learners.

Outlier Determination. Then, determining the outlier threshold is to help the learner decide whether to accept or reject an instance. In particular, each one-class learner has its own outlier threshold, independent of each other. The determination method is mainly

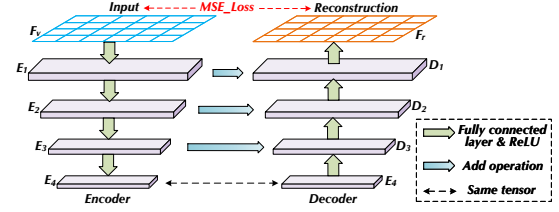


Figure 3: The AutoEncoder architecture in Trident.

based on the analysis of previous data, and the specific threshold is manually preset or obtained by probability and statistics.

Fine-Grained Label Assignment. The need for fine-grained labels includes both known and unknown classes. For known classes, the labels will be jointly derived from the results of multiple learners, as described above. Those samples that are not accepted by any learner refer to unknown classes and will enter the buffer. In the buffer, all unknown classes are assigned labels through a series of unsupervised strategies. Finally, samples are labeled as specific known types or fine-grained unknown classes. Noteworthy, these detected unknown classes can be used to build new one-class learners, given that Trident supports class increments.

4 TSIEVE: ONE-CLASS LEARNING

In this section, we provide three model architectures to perform the one-class learning task. Each model architecture corresponds to its unique feature vector extraction, but all are based on the traffic session with the 5-tuple index, i.e., $\{Source\ IP, Source\ Port, Destination\ IP, Destination\ Port, Protocol\}$.

4.1 AutoEncoder

Feature Extraction. For AutoEncoder, the generated feature vector (for each bi-directional flow) aims to characterize the traffic in terms of temporal, volumetric, and header-field distributions. Specifically, including (i) the protocol coding with one dimension. (ii) The count of a series of flags from layers 1-4 (such as IP Fragment, TCP Flags) in three situations (forward, backward, and bidirectional). (iii) The statistical results (i.e., *max*, *min*, *mean*, and *std*) for several transport-functional fields (e.g., time to live, window size, and time interval) in three situations about direction. In general, all features are either *int* or *float* types.

Model Architecture. A vanilla design is directly using the typical AutoEncoder (AE) as the one-class learner. However, deep layers in this typical architecture may lose some feature information. To this end, we add the tensor from the encoder to the corresponding layer in the decoder, so as to preserve more semantic information in different tiers. The model architecture is shown in Fig. 3, which is somewhat similar to U-Net [39] (a convolutional network). It contains an encoder and a decoder, which are symmetrical to each other. For example, in the encoder, the d -dimension input is concatenated with four fully connected layers (with the ReLU activation), in which the corresponding feature vectors are $F_v \rightarrow E_1 \rightarrow E_2 \rightarrow E_3 \rightarrow E_4$ and the dimensions refer $d \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32$. Notably, the decoder part will cascade the intermediate output of the encoder for addition operation (denote as the blue arrow in Fig. 3). For instance, D_2 can be calculated by $D_2 = ReLU(Linear(D_3 + E_3))$. Such an addition operation preserves richer features at each level to alleviate information loss during reconstruction. Meanwhile, we employ

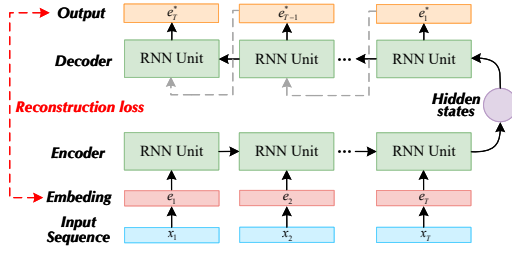


Figure 4: The RNN architecture in Trident.

mean squared error (MSE) as the loss function since the per-learner is essentially performing the data reconstruction. Empirically, a well-trained learner will output a smaller MSE loss for same-class samples, while data of different classes will lead to a larger loss.

4.2 Recurrent Neural Network

Features Extraction. For input to the Recurrent Neural Network (RNN) model, we extract the packet field from the raw traffic and expand the feature according to time to form sequences. Subsequently, all feature vectors are pruned or padded to T time steps. Per-packet feature vector consists of packet length, direction, the interval of arrival time, and values of fields in the packet header, *e.g.*, TTL (Time To Live), TCP flags, TCP window size, and SSL/TLS fields. Therefore, a sample can be denoted as $(x_1, x_2, \dots, x_T)$, where T is the time steps and $x_i \in \mathbb{R}^d$ is the d -dimensional packet feature at time step i . Such feature extraction is similar to most sequence-based methods, except that we focus on more fields (such as SSL/TLS) than just packet length and timestamp [30, 63].

Model Architecture. To accomplish data reconstruction as a one-class learner, we consider a representative Seq2Seq model [47]. The model architecture (more details in § B.1) is shown in Fig. 4, it consists of an encoder and a decoder of equal length. We cascade an *embedding layer* between the input and the RNN unit to reduce the impact of feature values, given different feature fields may have different value ranges. Formulaically, the x_i in each time step will go through the embedding layer to obtain e_i and then be fed to the encoder's RNN unit U_{RNN}^E . The hidden state output of the RNN unit is passed at different time steps. Afterward, the hidden state for the last time step of the encoder will be fed to the decoder as the initial state. The input to the decoder is the output of the previous time step (except for the first time step), with the hidden state passed. We collect the outputs e_i^* of decoder units U_{RNN}^D at all time steps. The reconstruction loss is still calculated with MSE, *i.e.*,

$$\mathcal{L} = \sum_{i=1}^T \sqrt{(e_i)^2 - (e_i^*)^2} \quad (1)$$

4.3 Graph Model

Features Extraction. The third model is a graph-based approach. To this end, we develop a novel session graph construction to extract features. As Fig. 12 displays, it considers the bidirectional traffic interaction between the client and server. We aggregate those adjacent packets in the same direction, forming an edge that includes the aggregated bytes. Based on such a process, the nodes are distributed on either the client or server. In the end, a bipartite graph can be obtained, which is characterized in that the points of the client and the client will not be connected, and the same applies to

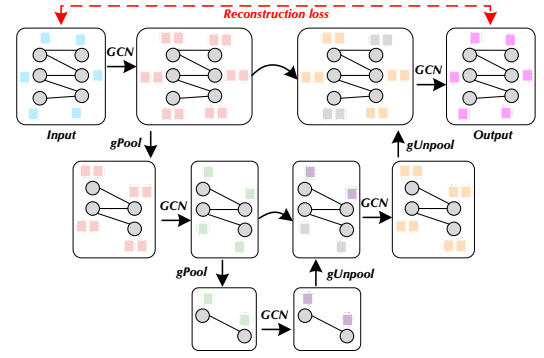


Figure 5: The GNN architecture in Trident.

the server. An example of packet aggregation can be found at the bottom of Fig. 12, two TCP payload packets with a length of 1,398 are aggregated to form an edge with 2,796 bytes. Thus, the feature vector of a node includes three elements, *i.e.*, [received bytes, sent bytes, duration]. The advantage of such data representation is that it has robustness, *e.g.*, it can alleviate the influence of changes from the Content Delivery Network (CDN) and Maximum Transmission Unit (MTU) on traffic analysis.

Model Architecture. For the above bipartite graph, we employ the U-Net-like GNN architecture [39], which is depicted in Fig. 5 (more details in § B.2). Among the model input, the traffic graph $\mathbb{G}$ can be represented by the adjacency matrix $A \in \mathbb{R}^{N \times N}$ and the feature matrix $X \in \mathbb{R}^{N \times C}$, where C and N represent the nodes' feature dimension and number, respectively. Also, each row vector $X_i = [x_i^0, x_i^1, \dots, x_i^C]$ in the feature matrix X denotes the feature vector of node i in the graph. During the encoder stage, the node feature vectors are first transformed into low-dimensional representations using a GCN layer. After that, two encoder blocks are performed, each of which contains a gPool layer [13] and a GCN layer. Correspondingly, there are two decoder blocks in the decoder part, and each block consists of a gUnpool layer [13] and a GCN layer. For blocks at the same level, the encoder block uses a skip connection to fuse the low-level spatial features from the encoder block. Finally, the output feature vectors $X_i' \in X'$ of nodes in the last layer are network embedding, which can be used for computing the reconstruction loss $\mathcal{L}$.

$$\mathcal{L} = \sum_{i=1}^N \sum_{j=1}^C \sqrt{(x_{ij}')^2 - (x_{ij})^2} \quad (2)$$

4.4 Incremental Update

Sample-Level Incremental Update. To adapt to new samples, we don't need to retrain the model, and only perform model generalization at the sample level. A strawman design might be to combine old data with new instances for model updates. The potential problem with this solution is that the scale of the data will continue to expand over time, resulting in inefficiency. Therefore, we could consider selecting some representative historical data. Recall the one-class learner reconstructs samples and computes an outlier threshold for the reconstruction loss. In order to keep the distribution of the reconstruction loss list as unchanged as possible before and after sampling, we sort the reconstruction loss list and sample them at intervals. For example, Fig. 6 (b)-(c) display the sampling strategy when setting sample rate $r=0.5, 0.2$ respectively, where red

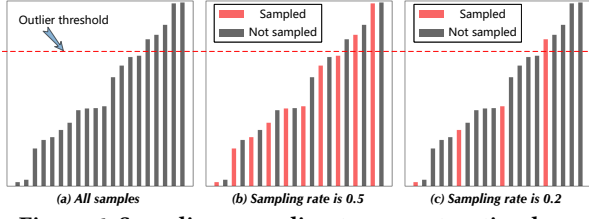


Figure 6: Sampling according to reconstruction loss.

columns represent sampled instances. In practice, the sampling rate can be set according to user requirements. These sampled historical data are used together with new data to incrementally update one-class learner parameters.

Class-Level Incremental Update. Since each one-class learner is independent of each other, the update operation can be conveniently performed. Using each new class to build the corresponding one-class learner and populate the learner list and threshold list, as shown in lines 1~7 in Algorithm 2. After updating, the new class can then be viewed as the known class. Overall, due to learners corresponding to classes one by one, building learners for new classes is incremental-manner. Meanwhile, the special advantage of this framework for model construction is that it can be run in parallel, easy to horizontal expansion, and reduce overhead, *e.g.*, distributed to deploy those one-class learners on different computing nodes.

5 TSCISSORS: OUTLIER IDENTIFICATION

In this section, we develop some outlier identification schemes such as the preset method and extreme value theory method.

Preset Method. A vanilla design is to preset the outlier ratio based on expert experience/knowledge. Given the output list L_o by a learner of length N_o , first sort it in ascending order to get L'_o . Consider a preset ratio R_p , which refers to treating the last $\text{int}(N_o \times R_p)$ data in L'_o as outliers. In other words, the threshold is the outlier boundary, *i.e.*, $L'_o[\text{int}(N_o \times R_p)]$. Actually, some similar work involving outlier detection methods are preset methods. For example, in the adversarial example detection landscape, many works construct a series of metrics to score inputs and use preset outlier thresholds to distinguish normal instances and adversarial examples [34, 54, 57, 64]. The advantage of the preset method is that it is convenient and prone to implement, but the limitation is that it needs to be manually set and sometimes lacks flexibility.

Extreme Value Theory Method. To solve this problem, we also leverage the Extreme Value Theory (EVT) [25] to estimate the outlier bounds. The original intention of this theory by statisticians is to study the possibility of extreme events. An elegant property of EVT is that the distribution of the extreme values is not dependent on the distribution of the data. In other words, these extreme events have the same kind of distribution, regardless of the original one, such as Frechet, Gamma, and Uniform distributions [43]. For specific details that utilize EVT to estimate thresholds, refer to § C.

6 TMAGNIFIER: LABEL ASSIGNMENT

We elaborate here on the process for fine-grained label assignment.

Known Class Classification. Algorithm 1 describes the procedure for label assignment. Consider a trained one-class learner list L_p and its threshold list L_t , each test sample will traverse each

Algorithm 1 Label Assignment

Require: The test dataset $\mathcal{S}$, the known-class one-class learner list L_p , and corresponding threshold list L_t , buffer $\mathcal{B}$

Ensure: y - class label for each $x \in \mathcal{S}$

```

1: Initialize prediction list  $Result = []$ , temporary lists  $R_k$  and  $R_u$ 
2: for  $x \in \mathcal{S}$  do
3:   Initialize reconstruction loss list  $L_r = []$ 
4:   for  $p_i \in L_p$  do
5:     Calculate reconstruction loss  $l \leftarrow p_i(x)$ 
6:     Append  $l$  to  $L_r$ 
7:   end for
8:   Candidate  $M_c = \text{array}(L_r)[\text{array}(L_r) < \text{array}(L_t)]$ 
9:   if  $\text{len}(M_c) == 0$  then
10:     $\mathcal{B} \leftarrow \mathcal{B} \cup x$ 
11:   else
12:     Select  $class_j$  s.t.  $L_r[class_j] == \min(M_c)$ 
13:     Append  $class_j$  label to  $R_k$ 
14:   end if
15: end for
16: Fine-grained new class results  $R_u \leftarrow \text{Clustering}(\mathcal{B})$ 
17:  $Result = R_k \cup R_u$ 
18: return  $Result$ 

```

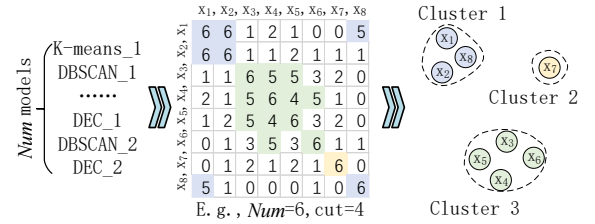


Figure 7: Unknown class identification.

learner to obtain reconstruction loss. If the reconstruction loss is less than the threshold, the class of this learner will be considered as a potential candidate (in line 8). Naturally, the instance that cannot match any of the learners is regarded as a new class (line 10). On the contrary, there is at least one learner that can cover the sample, *i.e.*, $\text{len}(M_c) \geq 1$, the class of the learner with the smallest reconstruction loss will be used as the prediction result (lines 12~13).

Unknown Class Identification. For new-class samples in the buffer pool $\mathcal{B}$, we leverage the clustering algorithm to perform fine-grained identification. Since we cannot know in advance how many unknown classes there are, a scheme such as K-means [17] that needs to set the number of clusters will be affected by this parameter. DBSCAN [7] is an optional solution. However, DBSCAN also needs to set parameters, such as *eps* or *min_samples*. Therefore, we intend to cluster with various parameters and integrate the results to achieve relatively stable results. The process of ensemble clustering (Num clustering models) is as follows. Considering the $\mathcal{B}$ sample set, we construct an adjacency matrix A_c to record the results of Num clustering algorithms. For example, if $A_c[i, j] = \Delta$, it refers to Δ clustering algorithms to cluster samples x_i and x_j into one cluster, where $x_i, x_j \in \mathcal{B}$ and $\Delta \leq Num$. As shown in Fig. 7, we need to generate the integrated results of the ensemble clustering according to A_c . The key is to determine a cut point to filter out the smaller values in the matrix, and thereby find the connected subcluster. Therefore, we can traverse the cut points from 0 to Num and calculate the silhouette coefficient [41], then select the cut point when the silhouette coefficient is the largest.

Table 1: The supervised evaluation results.

Dataset Model	USTC		Tor		IDS		CrossNet	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1
nPrint	95.68	89.75	99.50	93.37	99.86	99.62	93.89	91.58
FlowPic	76.88	73.57	94.82	89.62	99.22	99.24	74.38	72.45
FlowLens	86.91	84.34	98.76	91.38	98.55	91.09	83.22	81.41
ERNN	90.18	88.72	98.10	96.15	98.50	97.19	86.64	85.12
ET-BERT	94.51	90.26	95.69	92.70	99.93	99.63	91.22	88.45
Trident (AE)	95.65	91.20	94.13	89.01	99.70	99.29	92.30	90.61
Trident (RNN)	92.38	89.64	99.38	97.02	99.93	99.57	91.17	87.20
Trident (GNN)	94.32	90.07	97.30	94.51	99.62	99.34	94.51	92.36

7 EVALUATION

We comprehensively evaluate Trident, with code available online (repository <https://github.com/Secbrain/Trident/>).

7.1 Experiment Setup

Datasets. A series of public datasets are used for evaluation as follows. (i) **USTC-TFC2016** dataset [49] involves network traffic from two parts, *i.e.*, 10 types of malware and 10 categories of benign applications. (ii) **ISCXTor2016** dataset [24] includes diverse classes of Tor network traffic such as Email, Chat, FTP, and so on. (iii) **IDS2017&2018** datasets [9, 10] uses a proposed B-Profile system to generate the normal traffic, meanwhile, a series of intrusion behaviors are involved, including DoS/DDoS, Brute Force, Infiltration, *etc.* (iv) **CrossNet2021** dataset [27] contains traffic data from 20 categories of applications such as 360, Sougou, and CSDN in two practical scenarios, *i.e.*, stable and production networks.

Baselines. We use 16 models (involving supervised, unsupervised, and semi-supervised) as baselines, covering SOTA methods in the traffic analysis landscape. (i) Five supervised method: **nPrint** [18], **FlowPic** [42], **FlowLens** [3], **ERNN** [63], and **ET-BERT** [29]. (ii) Five binary classification anomaly detection: **Kitsune** [35], **Whisper** [12], **DeepLog** [6], **HyperVision** [11], and **Diff-RF** [33]. (iii) Six multi-classification anomaly detection: **K-means** [17], **DBSCAN** [7], **FlowPrint** [48], **Cls-Anomaly** [55], **SENC** [36], and **FARE** [28]. Details of these baselines can be found in Appendix D.

Metrics. Two popular benchmarks are used to evaluate the performance for identifying emerging classes [28], including the clustering accuracy (ACC) and adjusted mutual information (AMI). Their upper bounds are all 1 and the larger values mean the better effect. For supervised evaluation and binary classification anomaly detection (AD) experiments, we additionally calculate the Precision (*Pre*), Recall (*Rec*), F1-score, and the Area Under Curve (*AUC*).

Hyperparameter Settings. The sampling rate $r = 0.5$ and we also evaluate r in § 7.4. For outlier threshold determination, we use the EVT method by default, and traverse settings of the preset method to produce Receiver Operating Characteristic (ROC) curve in § 7.3. For unknown class clustering, three algorithms (K-means [17], DBSCAN [7], and DEC [52]) are used, by varying the hyper-parameters for each clustering algorithm to contribute $Num = 20$ models, and we vary Num to develop ablation experiments in § 7.4.

7.2 Supervised Evaluation

When all classes are known, Trident can directly construct per-class learner. Therefore, we first compare supervised SOTA methods to explore the known class classification effect of Trident. The classification results for the four datasets are summarized in Table 1,

Table 2: The binary classification anomaly detection results.

Dataset Model	USTC				IDS			
	ACC	Pre	Rec	F1	ACC	Pre	Rec	F1
Kitsune	92.45	92.57	92.23	92.41	93.07	93.52	92.49	93.07
Whisper	94.32	94.83	93.77	94.30	91.20	91.12	91.28	91.21
DeepLog	92.80	92.72	92.88	92.80	91.12	91.26	90.93	91.09
HyperVision	96.67	97.28	96.04	96.66	95.32	95.85	94.76	95.31
Diff-RF	93.55	94.36	92.72	93.53	92.25	92.98	91.43	92.19
Trident (AE)	97.25	97.26	97.29	97.27	95.45	95.09	94.91	95.00
Trident (RNN)	94.95	94.81	95.08	94.94	96.02	96.09	95.94	96.01
Trident (GNN)	96.45	96.73	96.21	96.47	93.55	93.31	93.76	93.53

including the accuracy and F1 score. In Tor and IDS datasets, Trident (RNN) presents the best performance. While Trident (AE) and Trident (GNN) are prominent in USTC and CrossNet datasets, respectively. Sometimes nPrint has a slight advantage in accuracy, but the difference is only $\sim 0.1\%$. Noteworthy, Trident achieves the superior F1 score, which indicates that Trident is hardly affected by data imbalance (in contrast, nPrint’s F1 score drops a lot). Overall, the classification performance of Trident for known classes is competitive compared to supervised SOTA methods.

7.3 Binary Classification Anomaly Detection

Then, we evaluate the binary classification anomaly detection methods. Since these methods only distinguish between “benign” and “abnormal”, we also change the output of Trident to “0” or “1” for a fair comparison, that is, not to distinguish the specific type of attack. Two attack-related datasets (USTC and IDS) are used to evaluate, only using their benign traffic to train for both the baseline and Trident. From Table 2, the detection results refer to Trident (AE) > HyperVision > Trident (GNN) > Trident (RNN) > Whisper > Diff-RF > DeepLog > Kitsune for USTC dataset and Trident (RNN) > Trident (AE) > HyperVision > Trident (GNN) > Kitsune > Diff-RF > Whisper > DeepLog for IDS dataset. This may be attributed to different attacks (datasets) corresponding to different optimal feature extraction and detection models. Nevertheless, Trident achieves the best binary anomaly detection performance in terms of accuracy, precision, recall, and F1 score. Moreover, we vary the detection threshold for each model to plot ROC curves in Fig. 13. According to Fig. 13 (a)-(b), we observe that Trident has fewer false positives while achieving a high true positive rate.

7.4 Multi-Classification Anomaly Detection

Class Increments. In this section, we evaluate the capability of Trident by varying $N_k:N_u$ (representing the number of known and unknown classes). We set $N_k:N_u = \{20:0, 16:4, 12:8, 8:12, 4:16, 0:20\}$ for USTC and CrossNet datasets and set $N_k:N_u = \{8:0, 6:2, 4:4, 2:6, 0:8\}$ for IDS and Tor datasets. The AMI results are shown in Fig. 8, and it is clear that as the number of unknown classes increases, Trident significantly outperforms other baselines. For instance, to detect the USTC dataset in Fig. 8 (a), when all classes are known, the performance gap is not very big, *e.g.*, the AMI of Trident is slightly higher than FlowPrint by $\sim 2\%$. However, with more unknown classes, the advantage of Trident becomes more obvious. When all classes are unknown, Trident outperforms other methods by $\sim 10\%$. Particularly, DBSCAN does not require training, and its detection results are similar in different known/unknown proportions, *i.e.*, always less than 60% of AMI.

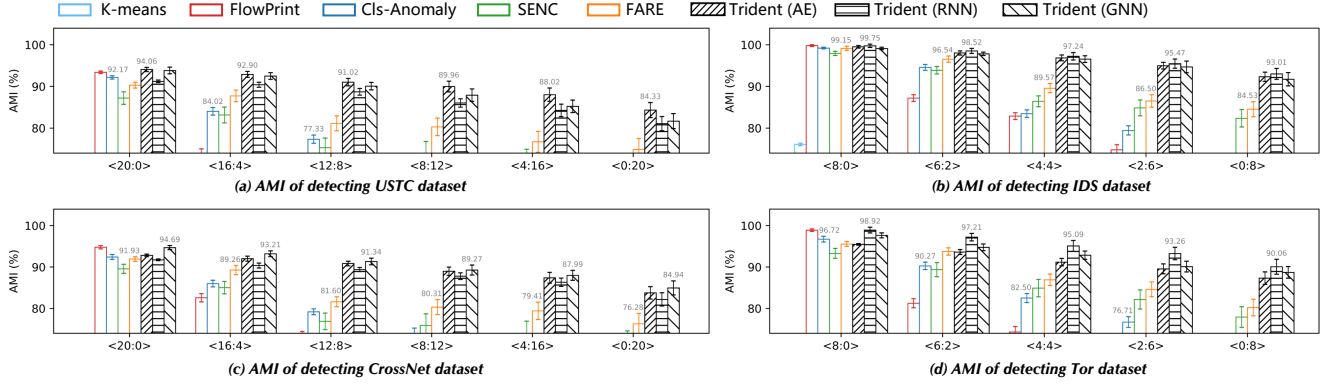


Figure 8: Evaluation in different known/unknown proportions.

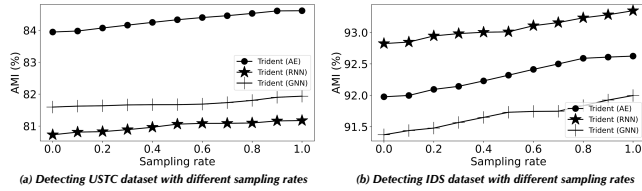


Figure 9: Multi-classification detection results of varying different sampling rates when performing sample increments.

Sample Increments. With $N_k = 0$, we vary the sampling rate r when performing sample-level incremental updates. The detection results for USTC and IDS datasets are summarized in Fig. 9. It is clear that as the sampling rate increases, the AMI results gradually increase. Therefore, if space overhead allows, we could tend to select more samples for updates.

Hyperparameter Evaluation. We also vary the cluster model Num to perform ablation experiments for $N_k = 0$. As Table 3 shown, from $Num = 5$ to $Num = 50$, the accuracy increases significantly. When $Num = 100$, the ACC change is relatively slight. Overall, Trident maintains solid performance with different parameter settings.

7.5 Concept Drift Evaluation

Concept drift is an unavoidable problem in anomaly detection systems, and we evaluate two scenarios *time bias* and *scenario bias*.

Time Bias. We conduct the time bias experiment by setting IDS2017 for training and using IDS2018 for testing, thereby exploring whether the traffic drifts over time will cause the failure of model detection. Fig. 10 (a)-(c) show the detection results for supervised, binary classification AD, and multi-classification AD models. In subfigure (a), we can see that ERNN is the best performer in the baseline because it considers network packet loss, retransmission, and out-of-order phenomena when building the model. Other supervised baselines are greatly affected by time bias. Nevertheless, our framework shows better robustness, e.g., Trident (RNN) is still able to achieve 96.71% ACC. For binary classification AD in subfigure (b), the impact of concept drift is less severe, since these are trained with only benign traffic in this setting and the impact on the model training process is limited. Trident still maintains the best performance under the binary classification AD setting. As for multi-classification AD, most of the models fail, with FARE guaranteeing an AMI of ~70% and others below 60%. In this setting, Trident realizes ~90% ACC and AMI, demonstrating its strong stability.

Scenario Bias. To develop the scenario bias evaluation, the Scenario A traffic is used for training and to detect Scenario B traffic from the CrossNet dataset. The two scenarios of the CrossNet dataset are collected from different network quality-of-service (QoS), such as different bandwidths and channel disturbance. The results are summarized in Fig. 10 (d)-(e). Similar to temporal bias, both the supervised and the multi-class AD models suffer to varying degrees. Also, all three model configurations of Trident outperform the baselines. Overall, the divide-and-conquer idea of Trident can effectively alleviate the impact of concept drift. In practice, we can also combine Trident with existing technologies [15] to deal with the problem of concept drift.

7.6 Overhead Evaluation

We measure the update time overhead for baseline algorithms and Trident, and the results are summarized in Fig. 11. All models run on the Ubuntu 18.04.2 server with Intel i7-12700K CPU, NVIDIA TITAN GPU, and 64 GB memory. Overall, nPrint, Whisper, Diff-RF, DBSCAN, K-means, and Hypervision are on one level ($<1ms$) since there are machine learning based methods. The deep learning model does have more time overhead (generally greater than $1ms$). For example, the time overhead of ERNN, DeepLog, and Trident (RNN) is relatively similar since they are all RNN-related models. The most time-consuming model is ET-BERT because it is essentially a large language model with massive parameters.

8 DISCUSSION AND LIMITATIONS

Model Selection. We designed Trident as a universal framework that could support various model architectures to characterize traffic. This paper mainly includes three types of models: AE, RNN, and GNN. We discuss here some model selection recommendations. Based on the experimental results in § 7, we observe that RNN is superior when the number of classes is small (e.g., IDS and Tor), but the performance will significantly decrease when there are more classes (refer to USTC and CrossNet datasets). When the training set and test set are identical distributions, AE is a good choice to deal with multiple categories. Compared with AE, GNN has better robustness even when there are concept drifts, this could be attributed as GNN's aggregation process alleviating the impact of changes from CDN and MTU on traffic (echo back § 4.3). Overall, users could choose desired model architectures or customize extensions/variants to cater to specific requirements.

Time bias: [train: IDS 2017] → [test: IDS 2018]						Scenario bias: [train: Scenario A] → [test: Scenario B]					
	ACC	F1	AUC	F1			ACC	F1	ACC	AMI	
nPrint	0.5575	0.4566	Kitsune	0.7528	0.7259	K-means	0.4673	0.4168	nPrint	0.4018	0.3119
FlowPic	0.4452	0.3013	Whisper	0.8221	0.8130	FlowPrint	0.5180	0.4720	FlowPic	0.2817	0.2412
FlowLens	0.6655	0.5909	DeepLog	0.8053	0.7913	Clis-Anomaly	0.5278	0.4863	FlowLens	0.5681	0.5589
ERNN	0.9013	0.8511	HyperVision	0.8641	0.8567	SENC	0.5791	0.5450	ERNN	0.7027	0.6831
ET-BERT	0.3378	0.2485	Diff-RF	0.6372	0.6081	FARE	0.7217	0.7068	ET-BERT	0.6626	0.6349
Trident (AE)	0.8790	0.8513	Trident (AE)	0.8509	0.8280	Trident (AE)	0.8390	0.8331	Trident (AE)	0.8836	0.8672
Trident (RNN)	0.9671	0.9522	Trident (RNN)	0.9327	0.9263	Trident (RNN)	0.9135	0.9020	Trident (RNN)	0.8070	0.8003
Trident (GNN)	0.9457	0.9360	Trident (GNN)	0.9165	0.9017	Trident (GNN)	0.8857	0.8698	Trident (GNN)	0.9192	0.9025

(a) Supervised (b) Binary classification AD (c) Multi-classification AD (d) Supervised (e) Multi-classification AD

Figure 10: The concept drift evaluation results.

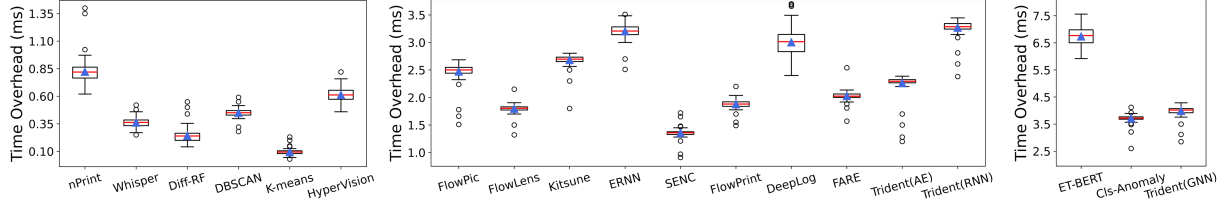


Figure 11: The time overhead.

Attack Category Recovery. When identifying fine-grained labels for unknown classes, it could occur to overestimate or underestimate the attack categories. The main reason refers to the extracted protocol features having different granularities. For example, some customers may need to distinguish between different HTTP flooding and some may not. Therefore, building a customized classification scheme in the output layer according to different needs may be beneficial to promoting Trident to widespread use. We will investigate these to advance the practicality of Trident.

Limitations and Future Work. First, different customers may require various detection granularity, the future work may consider a customized scheme, e.g., change the output layer of tSieve in Trident. Second, applying the automated feature extraction and model parameters tuning into Trident will lead in a good direction. Third, to provide customers with more reliable protection, the powerful adversary using a combination of multiple attacks needs to be further studied. Finally, in the real world, the external validation for detection results is practical and promising. We could work with practitioners to produce customized strategies, e.g., performing cross-validation with security vendors based on their analyzed results, threat intelligence, or discovered attack organizations.

9 RELATED WORK

Besides SOTA baselines in § 7.1, we list briefly some related work.

NIDS with Known Attack Classification. To classify known attacks, some works [42, 53, 65] design NIDSs based on statistical features by supervised learning methods [19, 37, 45, 58, 60], e.g., random forests, deep neural networks. Some other arts utilize Markov [23] or recurrent neural networks [30] to portray sequential features (e.g., packet length sequence). While these methods are less suitable for detecting unknown attacks.

NIDS with Unknown Attack Detection. These methods mainly involve three types of technologies: unsupervised, semi-supervised, and zero-shot learning. (i) *Unsupervised learning methods* such as clustering algorithms (e.g., K-means [17], DBSCAN [7], and CSPA [46]) have been applied to identify outliers in network traffic. They are also known as “zero-positive” learning [5, 16] due to solely using

benign samples for training. (ii) *Semi-supervised learning methods* such as Clis-Anomaly [55], FARE [28], and SENC [36] are usually composed of unsupervised and supervised learning. (iii) *Zero-shot learning methods* (ZSL) have been used to classify unknown classes in NIDS [38]. ZSL methods are non-incremental, and need rich “side information” to construct the feature mapping, which is not suitable for our problem. Overall, their focus is different from ours, Trident devotes to fine-grained unknown class detection and ever-changing traffic adaption in an incremental manner.

Some Recent Advances for NIDS. Security communities propose a series of advanced research directions including: solutions based on programmable switches [3, 32, 59] to adapt to high-speed bandwidth. Leveraging formal verification to analyze the security of NIDS [50, 51, 66]. And some research devoted the automated characterization [18] and interpretability for NIDS [16].

10 CONCLUSION

This paper presents Trident, a fine-grained traffic analysis framework towards identifying both known/unknown attack types, as well as adapting to variable traffic in an incremental manner. Based on our proposed framework, we implement three model architectures and extensively evaluate them on four public datasets. Moreover, we produce a series of experiments for Trident in terms of supervised, binary classification AD, and multi-classification AD. The results demonstrate the effectiveness and robustness of Trident outperforming existing SOTA methods.

ACKNOWLEDGMENTS

This work was supported in part by National Natural Science Foundation of China (62227805, 62072398, 62172405), by the Natural Science Foundation of Jiangsu Province (BK20220075), by the Fok Ying-Tung Education Foundation for Young Teachers in the Higher Education Institutions of China (No.20193218210004), by State Key Laboratory of Mathematical Engineering and Advanced Computing, and by Open Foundation of Henan Key Laboratory of Cyberspace Situation Awareness (HNTS2022001).

REFERENCES

- [1] August A. Balkema and Laurens De Haan. Residual life time at great age. *The Annals of probability*, pages 792–804, 1974.
- [2] Federico Barbero, Feargus Pendlebury, Fabio Pierazzi, et al. Transcending TRANSCEND: revisiting malware classification in the presence of concept drift. In *IEEE Symposium on Security and Privacy*, pages 805–823. IEEE, 2022.
- [3] Diogo Barradas et al. Flowlens: Enabling efficient flow classification for ml-based network security applications. In *NDSS*. The Internet Society, 2021.
- [4] Junyoung Chung, Çağlar Gülçehre, et al. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [5] Min Du, Zhi Chen, Chang Liu, Rajvardhan Oak, and Dawn Song. Lifelong anomaly detection through unlearning. In *CCS*, pages 1283–1297. ACM, 2019.
- [6] Min Du, Feifei Li, et al. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *CCS*, pages 1285–1298. ACM, 2017.
- [7] Martin Ester et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *KDD*, pages 226–231. AAAI Press, 1996.
- [8] FireEye. M-trends reports: Insights into today's breaches and cyber attacks. <https://content.fireeye.com/m-trends/rpt-m-trends-2020>, 2020.
- [9] Canadian Institute for Cybersecurity. Cse-cic-ids2018 on aws. [EB/OL], 2018. <https://www.unb.ca/cic/datasets/ids-2018.html> Accessed November 27, 2020.
- [10] Canadian Institute for Cybersecurity. Intrusion detection evaluation dataset (cicids2017). [EB/OL], 2018. <https://www.unb.ca/cic/datasets/ids-2017.html> Accessed November 27, 2020.
- [11] Chuanpu Fu et al. Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis. In *NDSS*. The Internet Society, 2023.
- [12] Chuanpu Fu, Qi Li, Meng Shen, and Ke Xu. Realtime robust malicious traffic detection via frequency domain analysis. In *CCS*, pages 3431–3446. ACM, 2021.
- [13] Hongyang Gao and Shuiwang Ji. Graph u-nets. In *ICML*, volume 97 of *Proceedings of Machine Learning Research*, pages 2083–2092. PMLR, 2019.
- [14] Scott D Grimshaw. Computing maximum likelihood estimates for the generalized pareto distribution. *Technometrics*, pages 185–191, 1993.
- [15] Dongqi Han et al. Anomaly detection in the open world: Normality shift detection, explanation, and adaptation. In *NDSS*. The Internet Society, 2023.
- [16] Dongqi Han, Zhiliang Wang, et al. DeepAID: Interpreting and Improving Deep Learning-based Anomaly Detection in Security Applications. In *CCS*, pages 3197–3217. ACM, 2021.
- [17] John A Hartigan and Manchek A Wong. A k-means clustering algorithm. *Journal of the royal statistical society. series c (applied statistics)*, 28(1):100–108, 1979.
- [18] Jordan Holland, Paul Schmitt, Nick Feamster, and Prateek Mittal. New directions in automated traffic analysis. In *CCS*, pages 3366–3383. ACM, 2021.
- [19] Guodong Huang et al. Efficient and low overhead website fingerprinting attacks and defenses based on TCP/IP traffic. In *WWW*, pages 1991–1999. ACM, 2023.
- [20] James Pickands III. Statistical inference using extreme order statistics. *The Annals of Statistics*, pages 119–131, 1975.
- [21] Peipei Jiang et al. Building in-the-cloud network functions: Security and privacy challenges. *Proc. IEEE*, 109(12):1888–1919, 2021.
- [22] Roberto Jordaney, Kumar Sharad, et al. Transcend: Detecting concept drift in malware classification models. In *USENIX Security Symposium*, pages 625–642. USENIX Association, 2017.
- [23] Maciej Korczynski and Andrzej Duda. Markov chain fingerprinting to classify encrypted traffic. In *INFOCOM*, pages 781–789. IEEE, 2014.
- [24] Arash Habibi Lashkari, Gerard Draper-Gil, Mohammad Saiful Islam Mamun, and Ali A. Ghorbani. Characterization of tor traffic using time based features. In *ICISSP*, pages 253–262. SciTePress, 2017.
- [25] Nicole A. Lazar. Statistics of extremes: Theory and applications. *Technometrics*, 47(3):376–377, 2005.
- [26] Hongda Li, Hongxin Hu, Guoifei Gu, Gail-Joon Ahn, and Fuqiang Zhang. vnids: Towards elastic security with safe and efficient virtualization of network intrusion detection systems. In *CCS*, pages 17–34. ACM, 2018.
- [27] Wenhao Li, Xiao-Yu Zhang, et al. Prograph: Robust network traffic identification with graph propagation. *IEEE/ACM Trans. Netw.*, pages 1–15, 2022.
- [28] Junjie Liang et al. FARE: enabling fine-grained attack categorization under low-quality labeled data. In *NDSS*. The Internet Society, 2021.
- [29] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Jing Yu. ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *WWW*, pages 633–642. ACM, 2022.
- [30] Chang Liu et al. Fs-net: A flow sequence network for encrypted traffic classification. In *INFOCOM*, pages 1171–1179. IEEE, 2019.
- [31] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *ICDM*, pages 413–422. IEEE Computer Society, 2008.
- [32] Zaoxing Liu et al. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric ddos attacks with programmable switches. In *USENIX Security Symposium*, pages 3829–3846. USENIX Association, 2021.
- [33] Pierre-Francois Marteau. Random partitioning forest for point-wise and collective anomaly detection - application to network intrusion detection. *IEEE Trans. Inf. Forensics Secur.*, 16:2157–2172, 2021.
- [34] Dongyu Meng and Hao Chen. Magnet: A two-pronged defense against adversarial examples. In *CCS*, pages 135–147. ACM, 2017.
- [35] Yisroel Mirsky et al. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *NDSS*. The Internet Society, 2018.
- [36] Xin Mu, Kai Ming Ting, and Zhi-Hua Zhou. Classification under streaming emerging new classes: A solution using completely-random trees. *IEEE Trans. Knowl. Data Eng.*, 29(8):1605–1618, 2017.
- [37] Sanghak Oh et al. Appsniffer: Towards robust mobile app fingerprinting against VPN. In *WWW*, pages 2318–2328. ACM, 2023.
- [38] Jorge Luis Rivero Pérez et al. A grassmannian approach to zero-shot learning for network intrusion detection. In *ICONIP (1)*, volume 10634 of *Lecture Notes in Computer Science*, pages 565–575. Springer, 2017.
- [39] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI (3)*, volume 9351 of *Lecture Notes in Computer Science*, pages 234–241. Springer, 2015.
- [40] Christian Rossow. Amplification hell: Revisiting network protocols for ddos abuse. In *NDSS*. The Internet Society, 2014.
- [41] Peter J Rousseeuw. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics*, 20:53–65, 1987.
- [42] Tal Shapira et al. Flowpic: Encrypted internet traffic classification is as easy as image recognition. In *INFOCOM Workshops*, pages 680–687. IEEE, 2019.
- [43] Alban Siffer, Pierre-Alain Fouque, et al. Anomaly detection in streams with extreme value theory. In *KDD*, pages 1067–1075. ACM, 2017.
- [44] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *IEEE Symposium on Security and Privacy*, pages 305–316. IEEE Computer Society, 2010.
- [45] Zhuoxue Song, Ziming Zhao, et al. I2RNN: An Incremental and Interpretable Recurrent Neural Network for Encrypted Traffic Classification. *IEEE Transactions on Dependable and Secure Computing*, 2023.
- [46] Alexander Strehl and Joydeep Ghosh. Cluster ensembles — A knowledge reuse framework for combining multiple partitions. *J. Mach. Learn. Res.*, 3:583–617, 2002.
- [47] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *NIPS*, pages 3104–3112, 2014.
- [48] Thijs van Ede et al. FlowPrint: Semi-Supervised Mobile-App Fingerprinting on Encrypted Network Traffic. In *NDSS*. The Internet Society, 2020.
- [49] Wei Wang et al. Malware traffic classification using convolutional neural network for representation learning. In *ICOIN*, pages 712–717. IEEE, 2017.
- [50] Zhongjie Wang et al. Symtcp: Eluding stateful deep packet inspection with automated discrepancy discovery. In *NDSS*. The Internet Society, 2020.
- [51] Zhongjie Wang et al. Themis: Ambiguity-aware network intrusion detection based on symbolic model comparison. In *CCS*, pages 3384–3399. ACM, 2021.
- [52] Junyuan Xie, Ross B. Girshick, and Ali Farhadi. Unsupervised deep embedding for clustering analysis. In *ICML*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 478–487. JMLR.org, 2016.
- [53] Fengli Xu et al. Understanding Mobile Traffic Patterns of Large Scale Cellular Towers in Urban Environment. *IEEE/ACM Trans. Netw.*, 25(2):1147–1161, 2017.
- [54] Weilin Xu, David Evans, and Yanjun Qi. Feature squeezing: Detecting adversarial examples in deep neural networks. In *NDSS*. The Internet Society, 2018.
- [55] Jian Yang et al. Conditional variational auto-encoder and extreme value theory aided two-stage learning approach for intelligent fine-grained known/unknown intrusion detection. *IEEE Trans. Inf. Forensics Secur.*, 16:3538–3553, 2021.
- [56] Limin Yang, Wenbo Guo, et al. CADE: detecting and explaining concept drift samples for security applications. In *USENIX Security Symposium*, pages 2327–2344. USENIX Association, 2021.
- [57] Yijun Yang, Ruiyuan Gao, Yu Li, Qiuxia Lai, and Qiang Xu. What you see is not what the network infers: Detecting adversarial examples based on semantic contradiction. In *NDSS*. The Internet Society, 2022.
- [58] Haozhen Zhang, Le Yu, et al. TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification. In *WWW*, pages 2066–2075. ACM, 2023.
- [59] Menghao Zhang, Guanyu Li, et al. Poseidon: Mitigating Volumetric DDoS Attacks with Programmable Switches. In *NDSS*. The Internet Society, 2020.
- [60] Ziming Zhao et al. CMD: Co-analyzed IoT Malware Detection Beyond the Network Traffic Domain. *IEEE Transactions on Mobile Computing*, 2023.
- [61] Ziming Zhao et al. Effective DDoS Mitigation via ML-Driven In-network Traffic Shaping. *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [62] Ziming Zhao, Zhaoxuan Li, et al. DDoS Family: A Novel Perspective for Massive Types of DDoS Attacks. *Comput. Secur.*, 2023.
- [63] Ziming Zhao, Zhaoxuan Li, et al. ERNN: Error-Resilient RNN for Encrypted Traffic Detection towards Network-Induced Phenomena. *IEEE Trans. Dependable Secur. Comput.*, 2023.
- [64] Ziming Zhao, Zhaoxuan Li, Tingting Li, et al. Poster: Detecting adversarial examples hidden under watermark perturbation via usable information theory. In *CCS*, pages 3636–3638. ACM, 2023.
- [65] Ziming Zhao, Zhaoxuan Li, Zhuoxue Song, and Fan Zhang. Work-in-progress: Towards real-time IDS via RNN and programmable switches co-designed approach.

In *RTSS*, pages 431–434. IEEE, 2023.

[66] Shitong Zhu, Shasha Li, et al. You do (not) belong here: detecting DPI evasion attacks with context learning. In *CoNEXT*, pages 183–197. ACM, 2020.

APPENDIX

A EXTENDED PROBLEM DEFINITIONS

At a high level, the generalized problem formulation refers to, given the training set involving $\{C_k^1, C_k^2 \dots C_k^n\}$ (n known classes), Trident will perform fine-grained classification for test set involving $\{C_k^1, C_k^2 \dots C_k^n, C_u^1, C_u^2 \dots C_u^m\}$ (n known classes + m unknown classes). The formulation can be further divided into three refined scenarios (B denotes “benign” and A denotes “attack”, the subscripts k and u correspond to “known class” and “unknown class”).

(i) *One type of benign + multiple types of attacks*. It refers to the training set involving $\{B, A_k^1, A_k^2 \dots A_k^n\}$, and the test set involving $\{B, A_k^1, A_k^2 \dots A_k^n, A_u^1, A_u^2 \dots A_u^m\}$, i.e., the statement in § 2.2, corresponding the IDS2017/2018 datasets in evaluations.

(ii) *Multiple types of benign + multiple types of attacks*. It refers to the training set involving $\{B_k^1, B_k^2 \dots B_k^x, A_k^1, A_k^2 \dots A_k^n\}$, and classes $\{B_k^1, B_k^2 \dots B_k^x, B_u^1, B_u^2 \dots B_u^y, A_k^1, A_k^2 \dots A_k^n, A_u^1, A_u^2 \dots A_u^m\}$ are in the test set, corresponding the USTC-TFC2016 dataset in evaluations.

(iii) *Multiple types of benign*. It refers to the training set involving $\{B_k^1, B_k^2 \dots B_k^n\}$, and classes $\{B_k^1, B_k^2 \dots B_k^n, B_u^1, B_u^2 \dots B_u^m\}$ are included in the test set, corresponding the CrossNet2021 and ISCX-Tor2016 datasets in evaluations.

B ADDITIONAL DETAILS OF TSIEVE

B.1 Additional Details for RNN Architecture

The Recurrent Neural Network (RNN) is a natural generalization of feedforward neural networks to sequences. Given a sequence of inputs $(x_1, x_2, \dots, x_T)$, a standard RNN computes a sequence of outputs $(y_1, y_2, \dots, y_T)$ by iterating the following equation:

$$h_t = \text{sigmoid}(W^{\text{hx}}x_t + W^{\text{hh}}h_{t-1}) \quad (3)$$

$$y_t = W^{\text{yh}}h_t \quad (4)$$

where W^{hx} , W^{hh} , and W^{yh} are the weight matrices of input-hidden, hidden-hidden, and hidden-output, respectively. The goal of the LSTM is to estimate the conditional probability $p(y_1, y_2, \dots, y_T | x_1, x_2, \dots, x_T)$, where $(x_1, x_2, \dots, x_T)$ is an input sequence and $(y_1, y_2, \dots, y_T)$ is its corresponding output sequence with the same length. The LSTM computes this conditional probability by first obtaining the fixed dimensional representation v of the input sequence $(x_1, x_2, \dots, x_T)$ given by the last hidden state of the LSTM, and then computing the probability of $(y_1, y_2, \dots, y_T)$ with a standard LSTM-LM formulation whose initial hidden state is set to the representation v of $(x_1, x_2, \dots, x_T)$:

$$p(y_1, y_2, \dots, y_T | x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(y_t | v, y_1, y_2, \dots, y_{t-1}) \quad (5)$$

Notably, as detailed in Section B.1, different from the above LSTM model, we only use the structure of sequence-to-sequence RNN model to construct the hidden layer relationship of the inputs itself, that is, the correct outputs and inputs are consistent. In this way, we employ the MSE as the loss function of the training process.

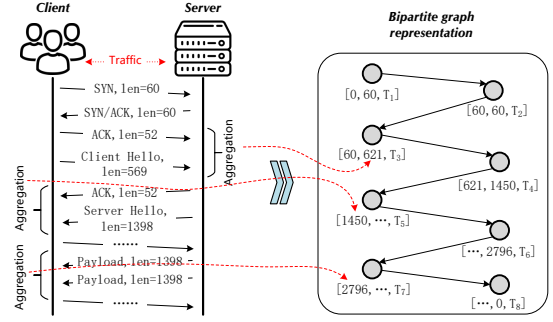


Figure 12: The bipartite graph representation.

B.2 Additional Details for GNN Architecture

As shown in Fig. 5 and detailed in Section 4.3, the GNN architecture contains Graph Pooling Layer, Graph Unpooling Layer, and GCN Layer.

Graph Pooling Layer. The Graph Pooling (gPool) layer is used to enable down-sampling on graph data. In this layer, a subset of nodes is adaptively selected to form a new but smaller graph. To this end, a trainable projection vector p is employed. By projecting all node features to 1D, k -max pooling for node selection can be performed. Since the selection is based on 1D footprint of each node, the connectivity in the new graph is consistent across nodes. Given a node i with its feature vector X_i , the scalar projection of X_i on p is $y_i = X_i p / \|p\|$. Among them, y_i measures how much information of node i can be retained when projected onto the direction of p . Specifically, the graph can be represented by two matrices; those are the adjacency matrix $A \in \mathbb{R}^{N \times N}$ and the feature matrix $X \in \mathbb{R}^{N \times C}$. Each non-zero entry in the adjacency matrix A represents an edge between two nodes in the graph. Each row vector X_i in the feature matrix X denotes the feature vector of node i in the graph. The layer-wise propagation rule of the graph pooling layer ℓ is:

$$y = X^\ell p^\ell / \|p\| \quad (6)$$

$$\text{idx} = \text{rank}(y, k) \quad (7)$$

$$\tilde{y} = y(\text{idx}) \quad (8)$$

$$\tilde{X}^\ell = X^\ell(\text{idx}, :) \quad (9)$$

$$A^{\ell+1} = A^\ell(\text{idx}, \text{idx}) \quad (10)$$

$$X^{\ell+1} = \tilde{X}^\ell \odot (\tilde{y} 1_C^T) \quad (11)$$

where k is the number of nodes selected in the new graph. $\text{rank}(y, k)$ is the operation of node ranking, which returns indices of the k -largest values in y . The idx returned by $\text{rank}(y, k)$ contains the indices of nodes selected for the new graph. $A^\ell(\text{idx}, \text{idx})$ and $X^\ell(\text{idx}, :)$ perform the row and/or column extraction to form the adjacency matrix and the feature matrix for the new graph. $y(\text{idx})$ extracts values in y with indices idx followed by a sigmoid operation. $1_C \in \mathbb{R}^C$ is a vector of size C with all components being 1, and $\odot$ represents the element-wise matrix multiplication.

Graph Unpooling Layer. To enable up-sampling operations on graph data, the graph unpooling (gUnpool) layer, which performs the inverse operation of the gPool layer and restores the graph into its original structure. To achieve this, we record the locations of nodes selected in the corresponding gPool layer and use this information to place nodes back to their original positions in the

Algorithm 2 Incrementally Add Learner

Require: The learner list L_p , the threshold list L_t , the new class label set $\mathcal{Y}$ and the corresponding sample set $\mathcal{X}$

Ensure: The updated learner list and the updated threshold list

```

1: for  $y \in \mathcal{Y}$  do
2:   Select subset  $b_i$  s.t.  $(x, y) \in b_i \in \mathcal{X}$ 
3:   Construct learner  $p_i$  based on  $b_i$ 
4:    $t_i \leftarrow \text{Outlier Threshold}(p_i, b_i)$ 
5:   Apend  $t_i$  to  $L_t$ 
6:   Apend  $p_i$  to  $L_p$ 
7: end for
8: return  $L_p, L_t$ 

```

graph. Formally, we propose the layer-wise propagation rule of graph unpooling layer as:

$$X^{\ell+1} = \text{distribute}(0_{N \times C}, X^\ell, \text{idx}) \quad (12)$$

where $\text{idx} \in \mathbb{Z}^{*k}$ contains indices of selected nodes in the corresponding gPool layer that reduces the graph size from N nodes to k nodes. $X^\ell \in \mathbb{R}^{k \times C}$ are the feature matrix of the current graph, and $0_{N \times C}$ are the initially empty feature matrix for the new graph. $\text{distribute}(0_{N \times C}, X^\ell, \text{idx})$ is the operation that distributes row vectors in X^ℓ into $0_{N \times C}$ feature matrix according to their corresponding indices stored in idx . In $X^{\ell+1}$, row vectors with indices in idx are updated by row vectors in X^ℓ , while other row vectors remain zero.

GCN Layer. Notably, there is a GCN layer before each gPool layer, thereby enabling gPool layers to capture the topological information in graphs implicitly. Before the processing of the GCN layer, the k -th graph power $\mathbb{G}^k$ to increase the graph connectivity. We employ $k = 2$ since there is a GCN layer before each gPool layer to aggregate information from its first-order neighboring nodes. Formally, the equation is delivered:

$$A^2 = A^\ell A^\ell, A^{\ell+1} = A^2(\text{idx}, \text{idx}) \quad (13)$$

where $A^2 \in \mathbb{R}^{N \times N}$ is the 2-th graph power. Now, the graph sampling is performed on the augmented graph with better connectivity. In the GCNs, the layer-wise forward-propagation operation is defined as:

$$X_{\ell+1} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{\frac{1}{2}} X_\ell W_\ell) \quad (14)$$

where $\tilde{A} = \tilde{A} + 2I$ is used to add self-loops in the input adjacency matrix A , X_ℓ is the feature matrix of layer ℓ . The GCN layer uses the diagonal node degree matrix $\tilde{D}$ to normalize $\tilde{A}$. W_ℓ is a trainable weight matrix that applies a linear transformation to feature vectors.

B.3 Additional Details of Class-Level Incremental Update

The pseudocode for class-level incremental update is described in Algorithm 2.

C ADDITIONAL DETAILS OF TSCISSORS

The Extreme Value Theory (EVT) [25] can be used to estimate the outlier bounds. Specifically, the form of Extreme Value Distributions (EVD) is presented as follows.

$$G_\gamma : x \mapsto \exp(-(1 + \gamma x)^{-\frac{1}{\gamma}}), \gamma \in \mathbb{R}, 1 + \gamma x > 0 \quad (15)$$

where γ denotes the extreme value index. An elegant property of EVT is that the distribution of the extreme values is not dependent on the distribution of the data. In other words, these extreme events have the same kind of distribution, regardless of the original one, such as Frechet, Gamma, and Uniform distributions [43].

For a trained learner, we can obtain the reconstruction loss list for its corresponding data subset. The upper quantile of the list is used as the initialization threshold t . Then, according to Pickands-Balkema-de Haan theorem (also called second theorem in EVT) [1, 20], the extrema of cumulative distribution function F converge in distribution to G_γ , if and only if a function σ exists, i.e.,

$$\bar{F}_t(x) = \mathbb{P}(X - t > x | X > t) \underset{t \rightarrow \tau}{\sim} \left(1 + \frac{\gamma x}{\sigma(t)}\right)^{-\frac{1}{\gamma}} \quad (16)$$

It means that $X - t$ (excess over threshold) tends to follow a Generalized Pareto Distribution (GPD)³ with parameters γ, σ . Once we get estimates $\hat{\gamma}, \hat{\sigma}$, the Peaks-Over-Threshold (POT) approach could be used to calculate the threshold as follows

$$\mathcal{T} \simeq t + \frac{\hat{\sigma}}{\hat{\gamma}} \left(\left(\frac{qn}{N_p} \right)^{-\hat{\gamma}} - 1 \right) \quad (17)$$

where q denotes the risk parameter, n refer to data size, and N_p represents the number of peaks.

In order to estimate $\hat{\gamma}, \hat{\sigma}$, the maximum likelihood estimation is considered an efficient method, and its goal is to maximizing (after logarithmic operation):

$$\log \mathcal{L}(\gamma, \sigma) = -N_p \log \sigma - \left(1 + \frac{1}{\gamma}\right) \sum_{i=1}^{N_p} \log \left(1 + \frac{\gamma}{\sigma} \mathcal{P}_i\right) \quad (18)$$

where $\mathcal{P} = \{l - t \text{ s.t. } l > t \text{ and } l \in L\}$ and L denotes the loss list. According to [14, 43], Grimshaw's proposal could be used to reduce the two variables optimization problem to a single variable equation. Specifically, if we get a solution (γ^*, σ^*) , the variable $x^* = \gamma^* / \sigma^*$ is solution of the scalar equation $u(x)v(x) = 1$ where:

$$u(x) = \frac{1}{N_p} \sum_{i=1}^{N_p} \frac{1}{1 + x \mathcal{P}_i} \quad v(x) = 1 + \frac{1}{N_p} \sum_{i=1}^{N_p} \log(1 + x \mathcal{P}_i) \quad (19)$$

That is to say, by finding a solution x^* of this equation, we can retrieve $\gamma^* = v(x^*) - 1$ and $\sigma^* = \gamma^* / x^*$. Finally, the estimates γ^* and σ^* can be used to calculate the threshold reference Eq. (17).

D ADDITIONAL DETAILS FOR EVALUATION

D.1 Additional Details for Baselines

We introduce and summarize the details for a series of baselines here (refer to § 7.1).

(i) Supervised Method.

- **nPrint** [18] is a tool that generates a unified packet representation and then leverages AutoML to fit the tabular data.
- **FlowPic** [42] processes packet length and timestamp fields and converts them into pictures and uses Convolutional Neural Networks (CNNs) to identify traffic.
- **FlowLens** [3] calculates statistical histograms of packet size distribution and adopts machine learning models (e.g., XGBoost) to perform classification.

³The location μ , the third parameter of GPD, is null in our case.

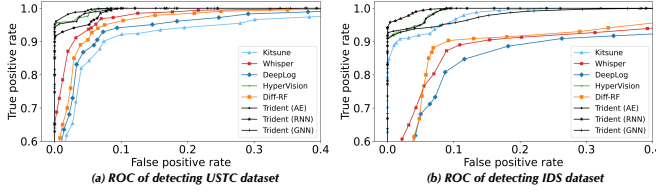


Figure 13: ROC of binary classification anomaly detection.

Table 3: The ACC (%) results with different *Num* settings.

Cluster model <i>Num</i>		5	10	20	50	100
Cross.	Trident (AE)	80.44	83.95	86.26	88.42	88.73
	Trident (RNN)	79.25	81.67	84.68	86.97	87.79
	Trident (GNN)	81.52	85.02	87.42	89.35	89.98
Tor	Trident (AE)	82.85	87.32	89.82	92.21	92.74
	Trident (RNN)	86.02	89.74	92.53	94.15	94.52
	Trident (GNN)	84.02	88.74	91.20	93.28	93.45

- **ERNN** [63] integrates the finite state automaton inside the RNN unit to cope with network-induced phenomena such as packet loss, retransmission, and out-of-order.
- **ET-BERT** [29] handles the raw packets in hexadecimal and deploys a pre-trained transformer to represent and learn the contextualized datagram-level information.

(ii) Binary Classification Anomaly Detection.

- **Kitsune** [35] calculates a series of statistical features and designs an ensemble of AutoEncoders to detection intrusions.
- **Whisper** [12] expresses traffic as frequency domain information through the fast Fourier transform and then performs robust identification.

- **DeepLog** [6] utilizes Long Short-Term Memory (LSTM) [4] to model the system log and detects the anomalies in sequences.
- **HyperVision** [11] is an unsupervised malicious traffic detection system that could capture flow interaction patterns represented by the graph's structural features.
- **Diff-RF** [33] takes into the frequencies of visits in the leaves on the isolated forest [31] basis to detect point-by-point and collective anomalies.

(iii) Multi-Classification Anomaly Detection.

- **K-means** [17] and **DBSCAN** [7] are typical density-based and distance-based unsupervised clustering algorithms respectively.
- **FlowPrint** [48] is a semi-supervised approach for fingerprinting mobile apps from encrypted network traffic.
- **Cls-Anomaly** [55] employs Conditioned Variational AutoEncoder and extreme value theory to devote multi-classification for known attacks.
- **SENC** [36] completes the semi-supervised classification based on isolation forest, yet it assumes only to emerge one unknown class at one time.
- **FARE** [28] is a semi-supervised clustering method for classification under low-quality labels. Note that it needs to specify the number of classes for FARE, we set it as ground truth.

D.2 Additional Evaluation Results

For binary classification anomaly detection results, we vary the detection threshold for each model to plot ROC curves in Fig. 13. The experiment results (for ISCXTor2016 and CrossNet2021 datasets) with different hyperparameter *Num* settings are summarized in Table 3.